FOOSUS

Alimentation Durable & Locale

Spécification des Conditions Requises pour l'Architecture

Approvisionnement Alimentaire Géographiquement Responsable

Projet : Approvisionnement Alimentaire Géographiquement Responsable | Client : Foosus

Information sur le document

Statut	VERSION PRÉLIMINAIRE — v1.0
Auteur	Architecte d'Entreprise, Fonction Architecture Foosus
Projet	Approvisionnement Alimentaire Géo-Conscient
Client	Foosus
Référencé dans	Déclaration de Travail d'Architecture — Projet Geo-Aware Food Sourcing
Framework	TOGAF 10 — Phase A & B (Vision & Architecture Business)
N° de Version	0.1
Date de Version	Mars 2026
Revu par	Natasha Jarson, CIO — Pete Parker, Responsable Ingénierie
Date de Révision	
Légende priorités	CRITIQUE = bloquant │ IMPORTANT = nécessaire │ STANDARD = souhaitable

Table des matières

Information sur le document **2**
Table des matières **3**
1. Objet de ce document **4**
2. Mesures du succès **5**
3. Conditions requises pour l'architecture **6**
3.1 Exigences Fonctionnelles 6
3.1.1 Géolocalisation et Recherche Géo-Consciente 6
3.1.2 Gestion des Utilisateurs et des Rôles 6
3.1.3 Processus de Commande et Panier 6
3.2 Exigences Non-Fonctionnelles 7
3.2.1 Performance et Scalabilité 7
3.2.2 Sécurité 7
3.2.3 Déploiement et Continuité 8
3.2.4 Observabilité et Expérimentation 8
3.2.5 Architecture des Données 9
4. Contrats de service business **10**
4.1 Accords de niveau de service (SLA) 10
4.2 Gestion des Non-Conformités 10
5. Contrats de service application **11**
5.1 Objectifs de niveau de service (SLO) 11
5.2 Indicateurs de niveau de service (SLI) 11
5.3 Conformité Structurelle — Microservices 11
5.4 Conformité Sécurité 12
5.5 Conformité Performance 12
5.6 Conformité Déploiement et Gouvernance 12
6. Lignes directrices pour l'implémentation **13**
6.1 Architecture Microservices 13
6.2 Approche Cloud-Native 13
6.3 Sécurité by Design 13
6.4 Livraison Continue 13
7. Spécifications pour l'implémentation **14**
7.1 Architecture des Services 14
7.2 Coexistence des Plateformes 14
8. Standards pour l'implémentation **15**
8.1 Standards Techniques 15
8.2 Standards de Qualité 15
8.3 Standards de Données 15
9. Conditions requises pour l'interopérabilité **16**
9.1 Intégrations Internes 16
9.2 Intégrations Externes (Vision Long Terme) 16
9.3 Standards d'Interopérabilité 16
10. Conditions requises pour le management du service IT **17**
10.1 Gouvernance Architecturale 17
10.2 Gestion des Incidents 17

10.3 Continuité de Service 17
11. Contraintes 18
11.1 Contraintes Budgétaires et Temporelles 18
11.2 Contraintes Organisationnelles 18
11.3 Contraintes Techniques 18
12. Hypothèses 19
Approbation 20

1. Objet de ce document

La Spécification des Conditions requises pour l'Architecture définit un ensemble d'exigences mesurables qui précisent ce qu'un projet d'implémentation doit accomplir pour être conforme à l'architecture cible.

Ce document constitue généralement une pièce centrale du contrat d'implémentation, ou d'un contrat visant une définition plus détaillée de l'architecture.

Il vient en complément du Document de Définition de l'Architecture, avec un rôle distinct mais complémentaire : là où le Document de Définition apporte une vision qualitative et exprime l'intention de l'architecte, la Spécification des Conditions requises propose une vision quantitative, fondée sur des critères concrets et vérifiables.

Son objectif est de définir de manière claire, complète et sans ambiguïté les exigences nécessaires à l'implémentation de la nouvelle architecture Foosus, ainsi que les critères permettant d'en valider la conformité. À ce titre, il sert de référence commune pour l'ensemble des équipes impliquées dans le projet.

Structuré selon le modèle TOGAF *Architecture Requirements Specification*, ce document couvre deux dimensions complémentaires :

- les exigences d'implémentation, qui décrivent ce que l'architecture doit permettre de construire ;
- les exigences de conformité, qui précisent comment s'assurer que l'architecture est bien respectée.

2. Mesures du succès

Les indicateurs suivants permettent de mesurer objectivement le succès de l'implémentation de la nouvelle architecture Foosus. Ces KPIs sont directement issus du Brief Business v1.1 et sont validés par l'équipe produit et la direction.

Indicateur	Valeur Actuelle	Objectif Cible
Nouvelles adhésions utilisateurs/jour	—	Augmentation de 10 %
Adhésions de producteurs alimentaires	1,4 / mois	4 / mois
Délai moyen de parution (lead time)	3,5 semaines	< 1 semaine
Taux d'incidents de production P1	> 25 / mois	< 1 / mois
Disponibilité plateforme	< 99%	99,9 % (< 9h/an)
Temps de réponse API (p95)	—	< 500 ms
Score Lighthouse mobile	—	> 85

** Le délai moyen de parution (lead time) est défini comme le temps écoulé entre la rédaction d'une user story et sa mise en production.*

3. Conditions requises pour l'architecture

Cette section détaille l'ensemble des exigences d'implémentation pour la nouvelle plateforme Foosus. Les exigences sont organisées par domaine fonctionnel et non-fonctionnel, chacune étant identifiée par un code unique, associée à une priorité et tracée jusqu'à sa source.

3.1 Exigences Fonctionnelles

3.1.1 Géolocalisation et Recherche Géo-Consciente

ID	Catégorie	Exigence	Priorité	Source
GEO-01	Géolocalisation	Le système doit permettre la détection de la position géographique de l'utilisateur (GPS ou IP) avec un degré de précision suffisant pour identifier les fournisseurs dans un rayon configurable	CRITIQUE	Brief Business v1.1, p.2
GEO-02	Calculateur distance	Un calculateur de distance doit permettre aux consommateurs de trouver les fournisseurs les plus proches par ordre croissant de distance	CRITIQUE	Brief Business v1.1, p.2
GEO-03	Recherche géo-spatiale	La recherche de fournisseurs doit prioriser la proximité géographique comme critère de tri principal, avec possibilité de filtrage par rayon	CRITIQUE	Brief Business v1.1, p.7
GEO-04	Visualisation cartographique	L'interface doit intégrer une visualisation cartographique des fournisseurs disponibles autour de la position de l'utilisateur	IMPORTANT	Wireframes UX (Brief p.8)
GEO-05	Statistiques produit	Chaque offre alimentaire doit afficher des informations statistiques secondaires (ex : indice glycémique) accessibles en consultation	STANDARD	Brief Business v1.1, p.8
GEO-06	Multi-région	L'architecture géo doit être extensible pour couvrir de nouvelles villes et régions sans refactorisation majeure	CRITIQUE	Brief Business v1.1, p.2

3.1.2 Gestion des Utilisateurs et des Rôles

ID	Catégorie	Exigence	Priorité	Source
USR-01	Rôles utilisateurs	La plateforme doit gérer trois types d'utilisateurs distincts avec des permissions spécifiques : Consommateurs, Fournisseurs alimentaires, Back-office	CRITIQUE	Brief Business v1.1, p.2
USR-02	Profil consommateur	Les consommateurs doivent pouvoir gérer un profil avec préférences alimentaires, historique et localisation sauvegardée	IMPORTANT	Brief Business p.2
USR-03	Profil fournisseur	Les fournisseurs doivent pouvoir gérer leur profil, catalogue d'offres, localisation et disponibilités	CRITIQUE	Brief Business p.2
USR-04	Back-office	Une interface back-office doit permettre la gestion administrative de la plateforme (validation fournisseurs, modération, analytics)	IMPORTANT	Brief Business p.2

3.1.3 Processus de Commande et Panier

ID	Catégorie	Exigence	Priorité	Source
CMD-01	Recherche offres	Le consommateur doit pouvoir rechercher et identifier des produits alimentaires par mots-clés, catégorie et filtres géo	CRITIQUE	Brief Business p.8
CMD-02	Panier d'achat	Le consommateur doit pouvoir ajouter des offres alimentaires à un panier et les modifier avant commande	CRITIQUE	Brief Business p.8
CMD-03	Paiement à la livraison	Le processus de commande initial doit supporter le paiement à la livraison comme mode principal	CRITIQUE	Brief Business p.9
CMD-04	Communication fournisseur	Le système doit envoyer automatiquement les instructions de livraison et la facturation de commission par e-mail au fournisseur	CRITIQUE	Brief Business p.9
CMD-05	Extensibilité paiement	L'architecture du paiement doit être extensible pour intégrer des prestataires tiers (Stripe, PayPal) sans refactorisation du service de commande	IMPORTANT	Brief Business p.9 (vision LT)
CMD-06	Messagerie intégrée	L'architecture doit préparer l'intégration future d'une messagerie interne entre consommateurs et fournisseurs	STANDARD	Brief Business p.9 (vision LT)

3.2 Exigences Non-Fonctionnelles

3.2.1 Performance et Scalabilité

ID	Catégorie	Exigence	Priorité	Source
PERF-01	Scalabilité élastique	La plateforme doit scaler automatiquement pour absorber des pics de charge jusqu'à 10x le trafic nominal sans dégradation de performance	CRITIQUE	Brief p.4
PERF-02	Temps de réponse	Le temps de réponse des API doit être inférieur à 500ms pour 95% des requêtes en conditions nominales	CRITIQUE	Brief p.4-5
PERF-03	Performance mobile	Les interfaces doivent être utilisables sur connexions cellulaires lentes (3G, 2G) avec un temps de chargement initial < 3 secondes	CRITIQUE	Brief p.2, p.5
PERF-04	Mode dégradé	En cas de surcharge, les utilisateurs connectés doivent pouvoir continuer à accéder aux services en mode dégradé (circuit breaker)	CRITIQUE	Brief p.4
PERF-05	Disponibilité 24/7	La plateforme doit garantir une disponibilité de 99,9% (< 9h d'indisponibilité/an), opérant 24h/24	CRITIQUE	Brief p.5
PERF-06	Performance géo	Les requêtes géo-spatiales (calcul de distance, recherche par rayon) doivent répondre en moins de 800ms pour 95% des cas	IMPORTANT	Brief p.2

3.2.2 Sécurité

ID	Catégorie	Exigence	Priorité	Source
SEC-01	Authentification	Toutes les APIs doivent être protégées par authentification OAuth2/OIDC. Les tokens JWT doivent avoir une durée de vie configurée	CRITIQUE	Brief p.4

ID	Catégorie	Exigence	Priorité	Source
SEC-02	Autorisation RBAC	Un système de contrôle d'accès basé sur les rôles (RBAC) doit être implémenté pour tous les services	CRITIQUE	Brief p.2
SEC-03	Chiffrement en transit	Toutes les communications doivent être chiffrées via TLS 1.3 minimum, sans exception	CRITIQUE	Principe P4
SEC-04	Chiffrement au repos	Les données personnelles et sensibles doivent être chiffrées au repos dans les bases de données	CRITIQUE	Principe P4 + RGPD
SEC-05	Sécurité by design	Chaque nouveau service doit passer une revue de sécurité avant mise en production (OWASP Top 10)	CRITIQUE	Brief p.4
SEC-06	Audit trail	Toutes les opérations sensibles (création de compte, modification de profil, commande) doivent être auditées et conservées	IMPORTANT	Principe P4
SEC-07	Protection données géo	Les données de localisation des utilisateurs doivent être traitées conformément au RGPD, avec consentement explicite	CRITIQUE	RGPD + Brief p.2

3.2.3 Déploiement et Continuité

ID	Catégorie	Exigence	Priorité	Source
DEP-01	Zéro downtime	Les déploiements en production doivent être effectués sans interruption de service (blue-green ou canary deployments)	CRITIQUE	Brief p.4
DEP-02	CI/CD automatisé	Un pipeline CI/CD doit automatiser les tests, la validation et le déploiement de chaque service indépendamment	CRITIQUE	Brief p.5
DEP-03	Feature flags	Un système de feature flags doit permettre d'activer/désactiver des fonctionnalités sans redéploiement	IMPORTANT	Brief p.9
DEP-04	Rollback rapide	Chaque déploiement doit pouvoir être annulé en moins de 5 minutes en cas d'incident critique	CRITIQUE	Brief p.4
DEP-05	Livraisons incrémentales	Chaque livraison doit être de taille réduite et ciblée (single responsibility releases)	CRITIQUE	Brief p.3, p.5
DEP-06	Environnements séparés	Des environnements de développement, staging et production strictement séparés doivent exister pour chaque service	CRITIQUE	Brief p.5

3.2.4 Observabilité et Expérimentation

ID	Catégorie	Exigence	Priorité	Source
OBS-01	Logging structuré	Tous les services doivent produire des logs structurés (JSON) centralisés dans un système d'agrégation	CRITIQUE	Principe P5
OBS-02	Distributed tracing	Chaque requête doit être tracée de bout en bout via un identifiant de corrélation pour le diagnostic d'incidents	CRITIQUE	Principe P5
OBS-03	Métriques business	Des métriques business (taux d'abandon, conversions, adhésions) doivent être collectées en temps réel	CRITIQUE	Brief p.6 (KPIs)

ID	Catégorie	Exigence	Priorité	Source
OBS-04	Alerting	Des alertes automatiques doivent être configurées pour les seuils critiques (disponibilité, latence, erreurs)	IMPORTANT	Principe P5
OBS-05	A/B Testing	L'infrastructure doit permettre l'exécution de tests A/B pour comparer des variantes de fonctionnalités sur des sous-ensembles d'utilisateurs	IMPORTANT	Brief p.6
OBS-06	Dashboards	Des tableaux de bord opérationnels et business doivent être accessibles aux équipes techniques et produit	STANDARD	Brief p.6

3.2.5 Architecture des Données

ID	Catégorie	Exigence	Priorité	Source
DATA-01	Isolation des données	Chaque microservice doit posséder sa propre base de données, sans accès direct aux données d'un autre service	CRITIQUE	Principe P2 + DDD
DATA-02	Migration sans downtime	Les migrations de schéma de base de données doivent être effectuées sans interruption de service (migrations progressives)	CRITIQUE	Brief p.4
DATA-03	Données géo-spatiales	Une base de données compatible avec les requêtes géo-spatiales (ex : PostGIS) doit être utilisée pour le service de géolocalisation	CRITIQUE	Brief p.2
DATA-04	Cache distribué	Un système de cache distribué (ex : Redis) doit réduire la charge sur les bases de données pour les données fréquemment accédées	IMPORTANT	Brief p.4
DATA-05	Event sourcing	Le service de commandes doit implémenter un pattern d'event sourcing pour la traçabilité et la réversibilité des opérations	IMPORTANT	Principe P1
DATA-06	RGPD compliance	Un mécanisme de suppression/anonymisation des données personnelles doit être implémenté (droit à l'oubli)	CRITIQUE	RGPD

4. Contrats de service business

4.1 Accords de niveau de service (SLA)

Les accords de niveau de service (SLA) définissent les engagements de la plateforme Foosus envers ses utilisateurs, qu'il s'agisse de consommateurs ou de fournisseurs, ainsi qu'envers les parties prenantes internes.

Ils précisent les niveaux de service attendus et servent de référence en cas d'écart. Tout non-respect d'un SLA entraîne le déclenchement d'une procédure de gestion des non-conformités.

Service	SLA	Condition
Disponibilité plateforme	99,9 %	< 9h d'indisponibilité non planifiée par an
Temps de réponse API (p95)	< 500 ms	Conditions nominales de charge
Temps de réponse API géo (p95)	< 800 ms	Requêtes géo-spatiales (calcul de distance, rayon)
Chargement interface mobile	< 3 secondes	Connexions cellulaires lentes (3G/2G)
Déploiements sans interruption	100 %	Déploiements en production via blue-green ou canary
Rollback en cas d'incident P1	< 5 minutes	Annulation complète d'un déploiement défaillant
Taux d'incidents P1	< 1 / mois	Objectif cible post-migration vers la nouvelle architecture

4.2 Gestion des Non-Conformités

Le processus suivant s'applique lorsqu'un critère de conformité architecturale n'est pas respecté :

Sévérité	Définition	Actions Requises	Délai de Résolution
CRITIQUE	Non-conformité compromettant la sécurité, la disponibilité ou les données utilisateurs	Blocage immédiat du déploiement + notification CIO + plan de remédiation sous 24h	< 48h
IMPORTANTE	Non-conformité impactant les principes architecturaux fondamentaux	Revue architecturale sous 5 jours + plan de correction	< 2 sprints
STANDARD	Déviation mineure des standards recommandés	Documentation dans le backlog technique + correction planifiée	< 1 trimestre

5. Contrats de service application

5.1 Objectifs de niveau de service (SLO)

Les objectifs de niveau de service définissent les cibles internes de qualité que chaque équipe s'engage à respecter pour ses services. Ces SLO sont mesurés en continu et reportés lors des revues architecturales mensuelles.

Objectif	Valeur Cible	Méthode de Mesure
Disponibilité globale	>= 99,9 %	Monitoring continu (Uptime Robot / Datadog)
Latence API p95	< 500 ms	APM / distributed tracing (Jaeger / Datadog)
Latence API géo p95	< 800 ms	Métriques spécifiques service géolocalisation
Score Lighthouse mobile	> 85	Audit automatisé CI/CD à chaque déploiement UI
Couverture tests	> 80 %	Rapport de couverture dans le pipeline CI/CD
Taux d'erreurs API	< 0,1 %	Monitoring des codes HTTP 5xx en production

5.2 Indicateurs de niveau de service (SLI)

Les indicateurs de niveau de service sont les métriques collectées en temps réel pour mesurer la performance réelle des services par rapport aux SLO définis.

Indicateur (SLI)	Outil de Collecte	Fréquence
Taux de disponibilité (uptime %)	Uptime monitoring (Datadog / CloudWatch)	Temps réel
Latence des requêtes API (p50, p95, p99)	APM avec distributed tracing	Temps réel
Taux d'erreurs (5xx / total requêtes)	Logs agrégés + alerting	Temps réel
Nombre d'adhésions utilisateurs/jour	Analytics plateforme	Quotidien
Délai moyen de parution (lead time)	Suivi CI/CD + tableau de bord livraison	Par sprint
Taux d'incidents P1 par mois	Système de gestion des incidents	Mensuel
Score Lighthouse mobile	CI/CD automatisé	À chaque déploiement UI

5.3 Conformité Structurelle — Microservices

ID	Critère de Conformité	Méthode de Vérification	Fréquence
CONF-S01	Chaque service possède son propre dépôt de code et son propre pipeline de déploiement	Audit dépôt GitHub, vérification des pipelines CI/CD	À chaque création de service
CONF-S02	Aucun service n'accède directement à la base de données d'un autre service	Revue de code automatisée + tests d'architecture (ArchUnit)	À chaque PR/merge
CONF-S03	Toute communication inter-services passe par des APIs documentées (OpenAPI 3.x)	Validation du contrat OpenAPI dans CI/CD	À chaque déploiement
CONF-S04	Les services ne partagent pas de librairies de domaine métier	Analyse des dépendances dans CI/CD	Hebdomadaire
CONF-S05	Chaque service expose des endpoints de health check (/health, /ready)	Tests automatisés dans CI/CD	À chaque déploiement

5.4 Conformité Sécurité

ID	Critère de Conformité	Méthode de Vérification	Fréquence
CONF-SEC01	100% des endpoints sont protégés par authentification (OAuth2/JWT)	Tests de sécurité automatisés (OWASP ZAP)	À chaque déploiement
CONF-SEC02	Aucune donnée sensible n'est exposée dans les logs ou les réponses d'API	Analyse statique du code + audit mensuel	Mensuel
CONF-SEC03	Toutes les communications utilisent TLS 1.3 minimum	Scan automatisé des certificats et configurations TLS	Hebdomadaire
CONF-SEC04	Les vulnérabilités OWASP Top 10 sont adressées avant toute mise en production	SAST/DAST dans le pipeline CI/CD	À chaque build
CONF-SEC05	Les dépendances tierces ne contiennent pas de vulnérabilités connues (CVE critique)	Scan automatique des dépendances (Snyk/Dependabot)	À chaque build

5.5 Conformité Performance

ID	Critère de Conformité	Méthode de Vérification	Fréquence
CONF-P01	95% des requêtes API répondent en moins de 500ms en charge nominale	Tests de charge automatisés (k6/JMeter) dans CI/CD staging	À chaque déploiement en staging
CONF-P02	La plateforme supporte 10x le trafic nominal sans dégradation	Tests de scalabilité trimestriels	Trimestriel
CONF-P03	Le score Lighthouse mobile est supérieur à 85 pour toutes les interfaces	Audit Lighthouse automatisé dans CI/CD	À chaque déploiement UI
CONF-P04	Le circuit breaker est activé et testé pour chaque service critique	Tests d'injection de fautes (chaos engineering)	Mensuel

5.6 Conformité Déploiement et Gouvernance

ID	Critère de Conformité	Méthode de Vérification	Fréquence
CONF-D01	Toute décision architecturale significative est documentée dans un ADR dans le dépôt GitHub	Revue du dépôt d'architecture par l'architecte	Bi-hebdomadaire
CONF-D02	Aucun déploiement en production ne cause d'interruption de service	Monitoring en production lors de chaque déploiement	À chaque déploiement
CONF-D03	La couverture de tests automatisés est supérieure à 80% pour chaque service	Rapport de couverture dans CI/CD	À chaque build
CONF-D04	Les migrations de bases de données sont réversibles (up/down migrations)	Revue de code obligatoire + test en staging	À chaque migration
CONF-D05	Les KPIs architecturaux sont mesurés et reportés mensuellement à la CIO	Dashboard de métriques architecturales	Mensuel

6. Lignes directrices pour l'implémentation

Les lignes directrices suivantes encadrent les décisions d'implémentation à tous les niveaux. Elles sont issues des principes architecturaux définis dans la Déclaration de Travail d'Architecture et doivent être respectées par toutes les équipes.

6.1 Architecture Microservices

Chaque domaine métier est isolé dans un microservice indépendant, notamment Géolocalisation, Utilisateurs, Commandes, Catalogue et Notifications.

Les échanges entre services se font exclusivement via des API REST documentées (OpenAPI 3.x) ou par événements asynchrones via un broker de messages.

Aucun service n’accède directement aux bases de données d’un autre : chaque microservice est propriétaire de ses données.

6.2 Approche Cloud-Native

Les services sont conçus pour être conteneurisés (Docker) et orchestrés via Kubernetes ou une solution équivalente.

L’infrastructure est gérée en tant que code (Infrastructure as Code) afin d’assurer la reproductibilité des environnements.

Le cloud est privilégié pour l’hébergement. Le choix final entre cloud et on-premise sera basé sur des critères objectifs de coût et de performance.

6.3 Sécurité by Design

La sécurité est intégrée dès la conception et non ajoutée a posteriori.

Chaque service fait l’objet d’une revue basée sur l’OWASP Top 10 avant sa mise en production.

Les données personnelles et de géolocalisation sont traitées conformément aux exigences du RGPD.

6.4 Livraison Continue

Chaque service dispose de son propre pipeline CI/CD, permettant des déploiements sans interruption de service.

Les mises en production s’appuient sur des stratégies de type blue-green ou canary afin de limiter les risques.

Les feature flags permettent d’activer progressivement les nouvelles fonctionnalités.

7. Spécifications pour l'implémentation

Les spécifications d'implémentation complètent les exigences en fournissant des orientations techniques plus précises sur l'implémentation attendue.

7.1 Architecture des Services

- **Service de géolocalisation**
  Ce service s'appuie sur une base de données géospatiale, avec une recommandation pour PostGIS. Il expose une API REST permettant notamment la recherche par rayon ainsi que le calcul de distances. Les résultats de recherche fréquents sont optimisés grâce à un cache distribué, par exemple via Redis.

- **Service utilisateurs**
  La gestion des identités repose sur OAuth2 / OIDC afin de garantir un haut niveau de sécurité et d'interopérabilité. Un contrôle d'accès basé sur les rôles (RBAC) est mis en place pour les trois profils d'utilisateurs : consommateurs, fournisseurs et back-office. Les données personnelles sont chiffrées au repos afin de renforcer la protection des informations sensibles.

- **Service commandes**
  Le service commandes est conçu autour de l'event sourcing afin d'assurer une traçabilité complète des actions. Il prend en charge le paiement à la livraison et reste extensible pour intégrer des prestataires de paiement tiers. Des notifications automatiques sont envoyées par e-mail aux fournisseurs à chaque nouvelle commande.

7.2 Coexistence des Plateformes

Pendant la phase de transition, la nouvelle architecture coexiste avec la plateforme existante selon les principes suivants :

- Les early adopters accèdent aux nouvelles fonctionnalités de recherche tout en conservant le processus de paiement existant.
- Aucun nouveau développement n'est effectué sur l'ancienne plateforme (mode maintenance uniquement).
- La migration des utilisateurs vers la nouvelle plateforme est proportionnelle à l'évolution des fonctionnalités disponibles.

8. Standards pour l'implémentation

Les standards suivants s'appliquent à l'ensemble des services de la nouvelle plateforme Foosus. Toute dérogation doit être documentée dans un Architecture Decision Record (ADR) et validée par l'architecte.

8.1 Standards Techniques

- **APIs** : utilisation de REST avec documentation via OpenAPI 3.x pour toutes les APIs inter-services et publiques.
- **Authentification** : OAuth2 / OIDC avec des tokens JWT à durée de vie configurée.
- **Chiffrement** : TLS 1.3 pour les communications en transit et AES-256 pour les données au repos.
- **Conteneurisation** : Docker pour l'empaquetage des services et Kubernetes pour leur orchestration.
- **Observabilité** : logs structurés au format JSON, traces distribuées et métriques exposées via Prometheus.

8.2 Standards de Qualité

- Couverture de tests automatisés supérieure à 80 % pour chaque service.
- Revue de code obligatoire avec au moins un validateur par pull request ou merge request.
- Score Lighthouse mobile supérieur à 85 pour l'ensemble des interfaces utilisateur.

8.3 Standards de Données

- Isolation stricte des données : chaque microservice possède sa propre base de données.
- Migrations de schéma progressives, réversibles (up/down) et sans interruption de service.
- Conformité RGPD assurée, avec des mécanismes de suppression et d'anonymisation des données personnelles.

9. Conditions requises pour l'interopérabilité

La nouvelle plateforme Foosus doit s'intégrer avec plusieurs systèmes internes et externes. Les exigences d'interopérabilité suivantes s'appliquent :

9.1 Intégrations Internes

La plateforme doit assurer la coexistence avec le système existant. Une API Gateway partagée permet de router les requêtes entre l'ancienne et la nouvelle plateforme en fonction des fonctionnalités disponibles.

Un mécanisme de synchronisation sécurisé est mis en place pour les données utilisateurs, afin de garantir une cohérence entre les deux plateformes durant la phase de transition.

9.2 Intégrations Externes (Vision Long Terme)

Le service de commandes doit être conçu de manière extensible pour intégrer différents prestataires de paiement tels que Stripe, PayPal ou d'autres, sans nécessiter de refonte majeure.

De même, le service de géolocalisation doit pouvoir s'appuyer de façon interchangeable sur des fournisseurs de cartographie comme Google Maps ou OpenStreetMap.

Enfin, l'architecture doit anticiper l'intégration future d'un système de messagerie interne permettant la communication entre consommateurs et fournisseurs.

9.3 Standards d'Interopérabilité

Toutes les APIs sont documentées avec OpenAPI 3.x et versionnées (v1, v2, etc.).
Les événements asynchrones entre services respectent un schéma standardisé, tel que CloudEvents ou équivalent.
Les échanges de données géospatiales suivent le standard GeoJSON.

10. Conditions requises pour le management du service IT

Les exigences de management du service IT garantissent la bonne gouvernance et la continuité opérationnelle de la plateforme Foosus.

10.1 Gouvernance Architecturale

Des revues architecturales sont organisées toutes les deux semaines sous la responsabilité de l'architecte d'entreprise.

Toute décision architecturale significative doit être formalisée dans un Architecture Decision Record (ADR) et versionnée dans le dépôt GitHub.

Les indicateurs de performance (KPIs) architecturaux sont transmis chaque mois à la CIO, Natasha Jarson.

10.2 Gestion des Incidents

Le taux cible d'incidents critiques (P1) est inférieur à 1 par mois, contre plus de 25 actuellement.

Chaque incident P1 fait l'objet d'un post-mortem documenté et partagé dans le dépôt d'architecture.

En cas de déploiement défaillant, le rollback doit pouvoir être effectué en moins de 5 minutes.

10.3 Continuité de Service

Les mises en production doivent être réalisées sans interruption de service, via des stratégies blue-green ou canary uniquement.

Des tests de chaos engineering sont réalisés chaque mois afin de valider la résilience des mécanismes de protection (notamment les circuit breakers).

La disponibilité cible de la plateforme est fixée à 99,9 %, avec un fonctionnement assuré 24h/24 et 7j/7.

11. Contraintes

Les contraintes suivantes ont été identifiées et doivent être prises en compte dans toutes les décisions d'implémentation :

11.1 Contraintes Budgétaires et Temporelles

- Le budget alloué à la phase 1 couvre deux trimestres, mais seul le budget du premier trimestre est actuellement sécurisé.
- L'ancienne plateforme ne peut pas être arrêtée pendant la construction de la nouvelle, la continuité de service étant obligatoire.
- Les livraisons doivent être progressives et régulières afin de permettre une mise en production rapide du nouveau système

11.2 Contraintes Organisationnelles

- Les équipes restent fortement attachées à la plateforme existante. Il est donc important d'éviter les solutions de contournement consistant à ajouter de nouvelles fonctionnalités directement dans l'ancien système.
- L'organisation fonctionne en mode Agile (Kanban), ce qui implique de préserver l'autonomie des équipes et de maintenir un rythme de livraison soutenu.
- La décision entre hébergement cloud et on-premise n'est pas encore arrêtée et devra être prise sur la base de données objectives et mesurables.

11.3 Contraintes Techniques

- La nouvelle architecture doit coexister avec l'existant pendant toute la phase de transition.
- Aucun nouveau développement fonctionnel ne sera réalisé sur l'ancienne plateforme, qui sera maintenue uniquement en mode maintenance.
- Enfin, la dette technique de l'ancien système ne doit pas être reproduite ni transférée dans la nouvelle architecture.

12. Hypothèses

Les hypothèses suivantes ont été retenues pour l'élaboration de ce document. Elles devront être validées ou ajustées au fur et à mesure de l'avancement du projet :

- La plateforme existante sera maintenue en mode maintenance. Aucun nouveau développement fonctionnel ne sera réalisé sur celle-ci afin de concentrer les efforts sur la nouvelle architecture.
- La nouvelle architecture sera conçue à partir des technologies actuelles, tout en restant suffisamment flexible pour intégrer de nouvelles technologies lorsque cela sera pertinent.
- La première phase reposera sur la coexistence des deux plateformes, avec une migration progressive des utilisateurs vers la nouvelle solution. Cette montée en charge sera corrélée à l'évolution des fonctionnalités disponibles.
- Les premiers utilisateurs pourront, par exemple, tester de nouvelles fonctionnalités comme la recherche intégrée au parcours de paiement tout en continuant à utiliser les fonctionnalités existantes.
- Une prise en compte précoce de la géolocalisation permettra d'ouvrir la voie à de futures innovations basées sur la localisation des utilisateurs et des fournisseurs.
- Une approche architecturale de type lean sera privilégiée afin de soutenir la feuille de route, préserver l'autonomie des équipes et maintenir un rythme de livraison rapide.
- Le choix entre hébergement cloud et on-premise sera basé sur des données objectives de coût et de performance, et non sur des préférences subjectives.
- La nouvelle architecture est pensée pour accompagner la croissance jusqu'au prochain million d'utilisateurs inscrits.